

C++ Notes

Templates

Adam HAMOUCH

May 2026

1. Définition

Un template est un **moule** : on écrit la logique une fois, le compilateur génère une version du code pour chaque type utilisé.

Les templates **n'existent pas à l'exécution** — tout est résolu à la compilation. Zéro overhead runtime.

Règle absolue : l'implémentation doit être dans le **header** (.h ou .hpp). Le compilateur a besoin du code complet au moment de l'instantiation. Un .cpp séparé = erreur de linker.

Pattern Handle typé : même logique, empêche de passer un `TextureID` là où un `MeshID` est attendu — erreur à la compilation, zéro overhead.

```
template<typename Tag>
struct Handle { uint32_t id; };
struct TextureTag {};
using TextureHandle = Handle<TextureTag>;
using MeshHandle = Handle<MeshTag>;
// uploadMesh(textureHandle) => ERREUR compile
```

2. Template de fonction

```
// T = parametre de type abstrait
template<typename T> // "typename" et "class" sont equivalents ici
T clamp(T value, T lo, T hi) {
    return value < lo ? lo : (value > hi ? hi : value);
}

// Le compilateur deduit T depuis les arguments
float s = clamp(velocity, 0.f, 100.f); // T = float
int h = clamp(hp, 0, 100); // T = int
// Specification explicite si deduction ambiguë :
clamp<float>(someInt, 0.f, 1.f);
```

Non-type parameter : on peut passer une **valeur** directement, pas seulement un type.

```
template<typename T, int N> // N est une valeur compile-time
struct RingBuffer {
    T data[N];
    int head = 0, tail = 0;
};
RingBuffer<DrawCall, 256> cmdBuf; // taille fixe, sur la stack
```

3. Template de classe

```
template<typename T>
class ComponentStorage {
public:
    void add(EntityID id, T& c) {
        idx[id] = data.size();
        data.push_back(std::move(c));
    }
    T& get(EntityID id) { return data[idx[id]]; }
private:
    std::vector<T> data;
    std::unordered_map<EntityID, size_t> idx;
};

// Une instance typee par composant
ComponentStorage<TransformComponent> transforms;
ComponentStorage<RenderComponent> renderers;
```

4. Spécialisation

Totale : implémentation différente pour un type exact.
Partielle : implémentation différente pour une famille de types (ex : tous les pointeurs).

```
// Version generique
template<typename T>
void serialize(T v, Buffer& b) { b.write(&v, sizeof(T)); }

// Specialisation totale pour bool
template<>
void serialize<bool>(bool v, Buffer& b) {
    uint8_t u = v ? 1 : 0; b.write(&u, 1);
}

// Specialisation partielle pour tous les pointeurs T*
template<typename T>
struct IsNetworked<T*> { static constexpr bool value = true; };
```

5. Templates avancés

Variadic — nombre indéfini de paramètres

```
// typename... Args = 0 ou N types quelconques
template<typename T, typename... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}

// make_unique<Mesh>(vertices, indices, name)
// => Args = {vector<Vertex>, vector<uint32_t>, string}
```

enable_if — invalider un template pour certains types

```
// Sans enable_if : erreur cryptique a l'instantiation
// Avec enable_if : "no matching function" au point d'appel
template<typename T,
        typename = std::enable_if_t<std::is_trivially_copyable_v<T>>>
void uploadGPU(T* data, size_t n) {
    memcpy(mappedPtr, data, n * sizeof(T)); // safe pour les POD
}

uploadGPU(vertices.data(), n); // OK : Vertex est POD
uploadGPU(meshes.data(), n); // ERREUR propre : Mesh a un unique_ptr
```

`<type_traits>` fournit les questions sur les types :
`is_arithmetic_v<T>`, `is_trivially_copyable_v<T>`,
`is_same_v<T,U>`, `is_base_of_v<Base,T>`...

C++20 — Concepts : remplacent `enable_if` par une syntaxe claire.

```
// C++17 enable_if (verbeux)
template<typename T,
        typename = std::enable_if_t<std::is_arithmetic_v<T>>>
T lerp(T a, T b, float t) { return a+(b-a)*t; }

// C++20 concept (lisible)
template<std::floating_point T>
T lerp(T a, T b, T t) { return a+(b-a)*t; }
```

if constexpr — branchement à la compilation

```
// Seule la branche vraie est compilée pour T donne
// L'autre peut être invalide pour T : pas d'erreur
template<typename T>
void serialize(T& val, Buffer& buf) {
    if constexpr (std::is_trivially_copyable_v<T>)
        memcpy(buf.data(), &val, sizeof(T)); // uniquement si POD
    else
        val.serialize(buf); // uniquement si T a .serialize()
    // Un if normal compilerait les deux => erreur sur int
}
```

6. Cas d'usage engine

- **RHI** : `RHI<VulkanBackend>` — swap de backend sans toucher au client.
- **ECS** : `ComponentStorage<T>` — un storage typé et cache-friendly par composant.
- **Resource manager** : `ResourceManager<Texture>` / `<Mesh>` — cache générique.
- **Handle typé** : `Handle<TextureTag>` — sécurité de type à la compilation.
- **Upload GPU** : `enable_if + is_trivially_copyable` pour autoriser uniquement les POD.